Vue3开发课程笔记总结



目录

1. Vue3基础概念
2. 组合式API
3. 响应式系统
4. 组件系统
5. 模板语法
6. 路由管理
7. 状态管理
8. 性能优化
9. TypeScript支持
10. 测试与部署

---

1. Vue3基础概念

1.1 Vue3简介

Vue.js 3.0（代号：One Piece）是Vue.js的最新主要版本，于2020年9月正式发布。主要特点：

- **性能提升**：渲染性能提升55%，更新性能提升133%

- **包体积优化**：核心包体积减少41%
- **TypeScript支持**：原生TypeScript支持
- **组合式API**：提供更灵活的逻辑复用方式
- **Fragment支持**：组件可以返回多个根节点

1.2 Vue3 vs Vue2对比

特性	Vue2	Vue3	改进说明
响应式系统	Object.defineProperty	Proxy	支持更多数据类型，性能更好
生命周期	beforeCreate/created	setup	统一的入口点
组件通信	props/emits	props/emits + provide/inject	增加provide/inject
模板语法	限制较多	支持多个根节点	Fragment支持
包体积	~42KB	~25KB	减少41%
TypeScript	第三方支持	原生支持	更好的类型推断

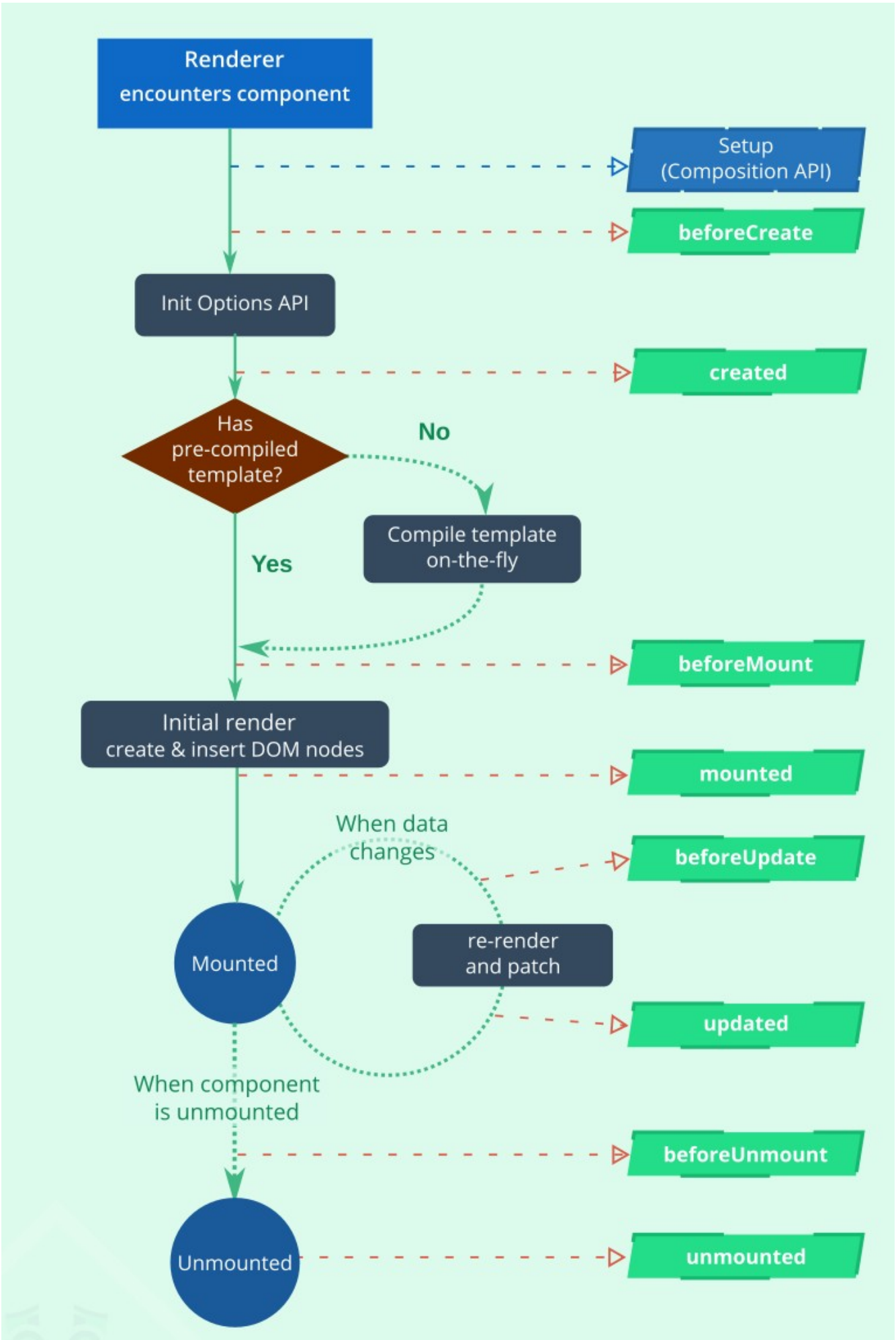
1.3 创建Vue3项目

```
# 使用Vite创建Vue3项目
npm create vue@latest my-vue3-app
cd my-vue3-app
npm install
npm run dev

# 或者使用Vue CLI
vue create my-vue3-app
```

---

2. 组合式API


Renderer
encounters component
Setup
(Composition API)
beforeCreate
Init Options API
created
Has
pre-compiled
template?
No
Compile template
on-the-fly
Yes
beforeMount
Initial render
create & insert DOM nodes
mounted
When data
changes
beforeUpdate
Mounted
re-render
and patch
updated
When component
is unmounted
beforeUnmount
Unmounted
unmounted
vuenoob.com

2.1 setup函数

setup是Vue3中新增的入口函数，用于组织组件的逻辑：

```
import { ref, reactive, computed } from 'vue'

export default {
  setup(props, context) {
    // 响应式数据
    const count = ref(0)
    const state = reactive({
      name: 'Vue3',
      version: '3.0'
    })

    // 计算属性
    const doubleCount = computed(() => count.value * 2)

    // 方法
    function increment() {
      count.value++
    }

    // 生命周期钩子
    onMounted(() => {
      console.log('组件已挂载')
    })

    // 返回模板需要的内容
    return {
      count,
      state,
      doubleCount,
      increment
    }
  }
}
```

2.2 响应式API详解

2.2.1 ref和reactive

API	适用场景	返回值	特点
`ref()`	基础类型数据	Ref对象	自动解包，支持.value
`reactive()`	对象/数组	代理对象	深度响应，递归监听
`readonly()`	只读数据	只读代理	防止修改
`toRefs()`	解构响应式对象	Ref对象	保持响应性

2.2.2 ref vs reactive对比

```
// ref示例
const count = ref(0)
console.log(count.value) // 0
count.value = 1
console.log(count.value) // 1

// reactive示例
const state = reactive({
  count: 0,
  user: {
    name: 'John',
    age: 30
  }
})
console.log(state.count) // 0
state.count = 1
console.log(state.count) // 1
state.user.age = 31 // 深度响应
```

2.3 计算属性与监听器

2.3.1 computed

```
import { ref, computed } from 'vue'

const count = ref(0)

// 只读计算属性
const doubleCount = computed(() => count.value * 2)

// 可写计算属性
const doubleCountWritable = computed({
  get: () => count.value * 2,
  set: (value) => {
    count.value = value / 2
  }
})
```

计算属性缓存机制详解
计算属性基于其依赖项进行缓存。当计算属性依赖的响应式数据发生变化时，计算属性会重新计算。 **缓存公式：**

$$缓存=\{\text{true,依赖数据未变化}\text{false,依赖数据发生变化}$$

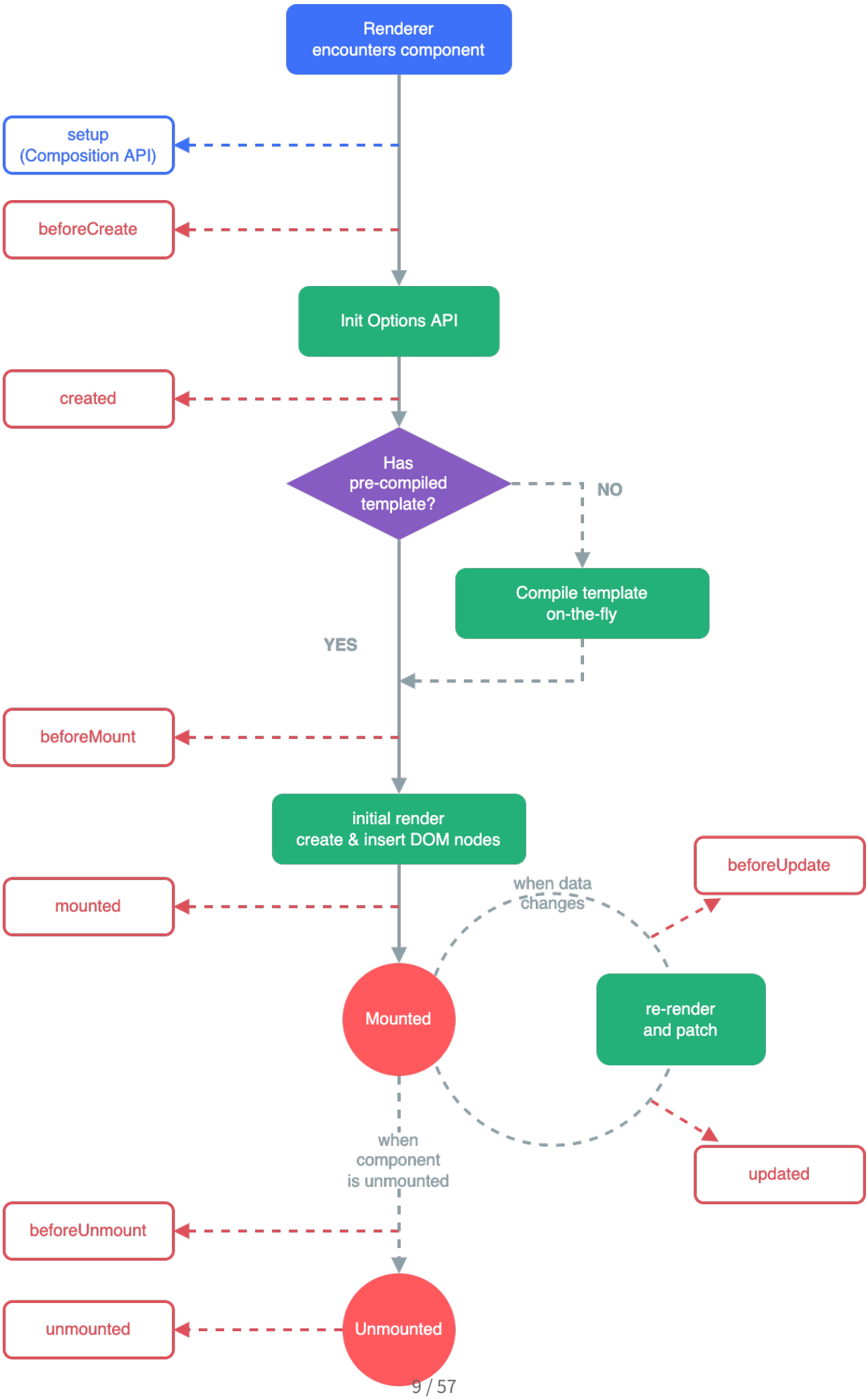
实现原理： 1. 计算属性内部维护一个dirty标志 2. 当依赖数据变化时，dirty设为true 3. 读取计算属性时，如果dirty为true则重新计算，否则返回缓存值

```
// 伪代码实现
class ComputedRefImpl {
  private _value: any
  private _dirty = true
  private effect: Effect

  get value() {
    if (this._dirty) {
      this._value = this.effect()
      this._dirty = false
    }
    return this._value
  }
}
```

2.4 生命周期钩子

Vue2生命周期	Vue3组合式API	执行时机
beforeCreate	setup	组件创建前
created	setup	组件创建后
beforeMount	onBeforeMount	组件挂载前
mounted	onMounted	组件挂载后
beforeUpdate	onBeforeUpdate	组件更新前
updated	onUpdated	组件更新后
beforeUnmount	onBeforeUnmount	组件卸载前
unmounted	onUnmounted	组件卸载后
errorCaptured	onErrorCaptured	错误捕获时
activated	onActivated	组件激活时
deactivated	onDeactivated	组件停用时


Renderer
encounters component
setup
(Composition API)
beforeCreate
Init Options API
created
Has
pre-compiled
template?
NO
Compile template
on-the-fly
YES
beforeMount
initial render
create & insert DOM nodes
mounted
when data
changes
beforeUpdate
Mounted
re-render
and patch
updated
when
component
is unmounted
beforeUnmount
unmounted
Unmounted

```
import {
  onMounted,
  onUpdated,
  onUnmounted
} from 'vue'

export default {
  setup() {
    onMounted(() => {
      console.log('组件已挂载')
    })

    onUpdated(() => {
      console.log('组件已更新')
    })

    onUnmounted(() => {
      console.log('组件已卸载')
    })
  }
}
```

---

3. 响应式系统

3.1 响应式原理

Vue3使用ES6的Proxy来实现响应式系统，相比Vue2的Object.defineProperty有以下优势：

响应式系统实现原理

Vue2的问题： 1. 无法检测新增/删除属性 2. 数组变化检测有限制 3. 嵌套对象需要递归监听 **Vue3的解决方案：** ![Vue3响应式系统依赖追踪图](/workspace/imgs/vue3-reactivity-dependency-tracking-diagram.jpg) 使用Proxy代理对象，拦截所有操作：

```
function reactive(target) {
  return new Proxy(target, {
    // 拦截读取操作
    get(target, key, receiver) {
      // 收集依赖
      track(target, key)
      return Reflect.get(target, key, receiver)
    },

    // 拦截设置操作
    set(target, key, value, receiver) {
      const result = Reflect.set(target, key, value, receiver)
      // 触发更新
      trigger(target, key)
      return result
    },

    // 拦截删除操作
    deleteProperty(target, key) {
      const result = Reflect.deleteProperty(target, key)
      trigger(target, key)
      return result
    }
  })
}
```

依赖收集和触发机制： 依赖收集公式：

$$依赖=组件实例+属性名+副作用函数$$

触发更新公式：

$$更新=找到依赖+执行副作用+重新渲染$$

3.2 响应式数据分类

数据类型	创建方式	特点	使用场景
基础类型	`ref()`	包装为Ref对象	数字、字符串、布尔值
对象类型	`reactive()`	深度响应	对象、数组

数据类型	创建方式	特点	使用场景
只读类型	`readonly()`	防止修改	外部传入的配置
计算属性	`computed()`	缓存机制	依赖其他响应式数据

3.3 响应式注意事项

3.3.1 浅层响应

```
import { shallowReactive, shallowRef } from 'vue'

// 浅层响应 - 只监听第一层属性变化
const shallowState = shallowReactive({
  user: {
    profile: {
      name: 'John'
    }
  }
})

// 修改深层属性不会触发更新
shallowState.user.profile.name = 'Jane' // ❌ 不会触发更新

// 修改第一层属性会触发更新
shallowState.user = { profile: { name: 'Jane' } } // ✅ 会触发更新
```

3.3.2 toRefs和toRef

```
import { reactive, toRefs, toRef } from 'vue'

const state = reactive({
  count: 0,
  name: 'Vue3'
})

// 将响应式对象转换为Ref对象，支持解构
const { count, name } = toRefs(state)

// 将对象属性转换为Ref
const countRef = toRef(state, 'count')
```

toRefs实现原理
toRefs将reactive对象的每个属性都转换为ref，保持响应性：

```
function toRefs(object) {
  const ret = {}
  for (const key in object) {
    ret[key] = toRef(object, key)
  }
  return ret
}

function toRef(object, key) {
  return {
    get value() {
      return object[key]
    },
    set value(val) {
      object[key] = val
    }
  }
}
```

为什么需要toRefs？ 当直接解构reactive对象时，会失去响应性：

```
// ❌ 错误做法 - 失去响应性
const { count } = reactive({ count: 0 })

// ✅ 正确做法 - 保持响应性
const { count } = toRefs(reactive({ count: 0 }))
```

---

4. 组件系统

4.1 组件基础

4.1.1 组件定义

```
// 单文件组件 (SFC)
<template>
  <div class="component">
    <h1>{{ title }}</h1>
    <button @click="increment">{{ count }}</button>
  </div>
</template>

<script setup lang="ts">
import { ref } from 'vue'

const props = defineProps<{
  title: string
  initialCount?: number
}>()

const emit = defineEmits<{
  increment: [value: number]
}>()

const count = ref(props.initialCount || 0)

function handleIncrement() {
  count.value++
  emit('increment', count.value)
}
</script>

<style scoped>
.component {
  padding: 20px;
}
</style>
```

4.1.2 组件通信

通信方式	适用场景	语法
props/emits	父子组件	向下传递/向上传递
provide/inject	跨层级	祖先提供/后代注入
v-model	双向绑定	语法糖
slot	内容分发	默认插槽/命名插槽

```
// 父组件
<template>
  <ChildComponent
    :message="parentMessage"
    @child-event="handleChildEvent"
    v-model="parentValue"
  />
</template>

// 子组件
<script setup lang="ts">
const props = defineProps<{
  message: string
  modelValue: string
}>()

const emit = defineEmits<{
  'update:modelValue': [value: string]
  'child-event': [data: any]
}>()

function sendToParent() {
  emit('child-event', { message: '来自子组件' })
}
</script>
```

4.2 插槽系统

4.2.1 默认插槽

```
// 父组件
<template>
  <Layout>
    <p>这是主要内容</p>
  </Layout>
</template>

// Layout组件
<template>
  <div class="layout">
    <header>Header</header>
    <main>
      <slot></slot>
    </main>
    <footer>Footer</footer>
  </div>
</template>
```

4.2.2 命名插槽

```
// 父组件
<template>
  <Layout>
    <template #header>
      <h1>页面标题</h1>
    </template>

    <template #default>
      <p>主要内容</p>
    </template>

    <template #footer>
      <p>页脚信息</p>
    </template>
  </Layout>
</template>

// Layout组件
<template>
  <div class="layout">
    <header>
      <slot name="header"></slot>
    </header>
    <main>
      <slot></slot>
    </main>
    <footer>
      <slot name="footer"></slot>
    </footer>
  </div>
</template>
```

4.2.3 作用域插槽

```
// 子组件
<template>
  <div>
    <slot :user="user" :items="items"></slot>
  </div>
</template>

<script setup lang="ts">
const user = { name: 'John', age: 30 }
const items = ['item1', 'item2', 'item3']
</script>

// 父组件
<template>
  <ChildComponent>
    <template #default="{ user, items }">
      <p>{{ user.name }} - {{ user.age }}</p>
      <ul>
        <li v-for="item in items" :key="item">{{ item }}</li>
      </ul>
    </template>
  </ChildComponent>
</template>
```

4.3 异步组件

```
import { defineAsyncComponent } from 'vue'

// 基础异步组件
const AsyncComponent = defineAsyncComponent(() =>
  import('./components/HeavyComponent.vue')
)

// 带加载和错误状态的异步组件
const AsyncComponentWithOptions = defineAsyncComponent({
  loader: () => import('./components/HeavyComponent.vue'),
  loadingComponent: LoadingComponent,
  errorComponent: ErrorComponent,
  delay: 200, // 加载延迟时间
  timeout: 3000 // 超时时间
})
```

---

5. 模板语法

5.1 插值语法

```
// 文本插值
<template>
  <div>{{ message }}</div>
  <div>{{ rawHtml }}</div>
  <div v-html="rawHtml"></div>
  <div>{{ number + 1 }}</div>
  <div>{{ ok ? 'YES' : 'NO' }}</div>
</template>

<script setup lang="ts">
const message = 'Hello Vue3'
const rawHtml = '<span style="color: red">红色文本</span>'
const number = ref(10)
const ok = ref(true)
</script>
```

5.2 指令系统

5.2.1 条件渲染

指令	作用	特点
`v-if`	条件渲染	真正的条件渲染
`v-else`	配合v-if	必须紧跟在v-if后
`v-else-if`	多重条件	必须在v-if后
`v-show`	显示/隐藏	切换display属性

```
<template>
  <!-- v-if条件渲染 -->
  <div v-if="type === 'A'">A</div>
  <div v-else-if="type === 'B'">B</div>
  <div v-else>C</div>

  <!-- v-show切换显示 -->
  <div v-show="isVisible">显示内容</div>
</template>
```

v-if vs v-show性能对比
性能公式：

初始渲染成本={v-if:高（需要创建/销毁DOM）v-show:低（仅切换display）

切换成本={v-if:低（已创建DOM）v-show:高（需要样式计算）

选择原则： - 初始加载：v-show性能更好 - 频繁切换：v-if性能更好 - 条件很少改变：v-if - 条件经常改变：v-show

5.2.2 列表渲染

```
<template>
  <!-- 遍历数组 -->
  <li v-for="(item, index) in items" :key="item.id">
    {{ index }} - {{ item.name }}
  </li>

  <!-- 遍历对象 -->
  <div v-for="(value, key, index) in object" :key="key">
    {{ index }}. {{ key }}: {{ value }}
  </div>

  <!-- 遍历数字 -->
  <span v-for="n in 10" :key="n">{{ n }}</span>
</template>

<script setup lang="ts">
const items = ref([
  { id: 1, name: 'Apple' },
  { id: 2, name: 'Banana' },
  { id: 3, name: 'Orange' }
])

const object = reactive({
  name: 'Vue3',
  version: '3.0',
  author: 'Evan You'
})
</script>
```

5.2.3 事件处理

```
<template>
  <!-- 基本事件 -->
  <button @click="handleClick">点击我</button>

  <!-- 事件参数 -->
  <button @click="handleClickWithParam($event, 'param')">
    带参数点击
  </button>

  <!-- 修饰符 -->
  <button @click.stop.prevent="handleClick">
    阻止事件传播和默认行为
  </button>

  <!-- 键盘事件 -->
  <input @keyup.enter="handleEnter" placeholder="按回车键">

  <!-- 鼠标事件 -->
  <div @click.middle="handleMiddleClick">中键点击</div>
</template>

<script setup lang="ts">
function handleClick() {
  console.log('按钮被点击')
}

function handleClickWithParam(event: Event, param: string) {
  console.log('事件:', event, '参数:', param)
}

function handleEnter() {
  console.log('回车键被按下')
}

function handleMiddleClick() {
  console.log('中键被点击')
}
</script>
```

5.3 样式绑定

5.3.1 class绑定

```
<template>
  <!-- 字符串绑定 -->
  <div :class="className">内容</div>

  <!-- 对象绑定 -->
  <div :class="{ active: isActive, 'text-danger': hasError }">
    内容
  </div>

  <!-- 数组绑定 -->
  <div :class="[activeClass, errorClass]">内容</div>

  <!-- 混合绑定 -->
  <div :class="[{ active: isActive }, className]">内容</div>
</template>

<script setup lang="ts">
const className = 'custom-class'
const isActive = ref(true)
const hasError = ref(false)
const activeClass = 'active'
const errorClass = 'text-danger'
</script>
```

5.3.2 style绑定

```
<template>
  <!-- 对象绑定 -->
  <div :style="{ color: activeColor, fontSize: fontSize + 'px' }">
    内容
  </div>

  <!-- 数组绑定 -->
  <div :style="[baseStyles, overridingStyles]">内容</div>
</template>

<script setup lang="ts">
const activeColor = ref('red')
const fontSize = ref(30)

const baseStyles = reactive({
  color: 'blue',
  fontWeight: 'bold'
})

const overridingStyles = reactive({
  fontSize: '20px',
  backgroundColor: 'yellow'
})
</script>
```

---

6. 路由管理

6.1 Vue Router 4

Vue Router 4是Vue3的官方路由库，提供以下特性：

- 完整的TypeScript支持
- 动态路由匹配
- 嵌套路由
- 编程式导航
- 路由守卫
- 懒加载支持

6.2 路由配置

```
// router/index.ts
import { createRouter, createWebHistory } from 'vue-router'
import type { RouteRecordRaw } from 'vue-router'

const routes: RouteRecordRaw[] = [
  {
    path: '/',
    name: 'Home',
    component: () => import('@/views/Home.vue'),
    meta: { requiresAuth: false }
  },
  {
    path: '/about',
    name: 'About',
    component: () => import('@/views/About.vue'),
    meta: { requiresAuth: false }
  },
  {
    path: '/user/:id',
    name: 'User',
    component: () => import('@/views/User.vue'),
    props: route => ({ userId: route.params.id }),
    meta: { requiresAuth: true }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: () => import('@/views/NotFound.vue')
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) {
      return savedPosition
    } else {
      return { top: 0 }
    }
  }
})
```

```
export default router
```

6.3 路由导航

```
<script setup lang="ts">
import { useRouter, useRoute } from 'vue-router'

const router = useRouter()
const route = useRoute()

// 编程式导航
function navigateToHome() {
  router.push('/')
}

function navigateWithParams() {
  router.push({
    name: 'User',
    params: { id: '123' },
    query: { name: 'John' }
  })
}

// 替换导航（不留下历史记录）
function replaceNavigation() {
  router.replace('/about')
}

// 前进后退
function goBack() {
  router.back()
}

function goForward() {
  router.forward()
}

// 获取当前路由信息
const currentPath = computed(() => route.path)
const currentQuery = computed(() => route.query)
</script>
```

6.4 路由守卫

```
// 全局前置守卫
router.beforeEach((to, from, next) => {
  const isAuthenticated = store.getters.isAuthenticated

  if (to.meta.requiresAuth && !isAuthenticated) {
    next('/login')
  } else {
    next()
  }
})

// 全局后置钩子
router.afterEach((to, from) => {
  // 页面访问统计
  analytics.track('page_view', { page: to.path })
})

// 路由独享守卫
const routes = [
  {
    path: '/admin',
    component: AdminPanel,
    beforeEnter: (to, from, next) => {
      const userRole = store.getters.userRole
      if (userRole === 'admin') {
        next()
      } else {
        next('/403')
      }
    }
  }
]

// 组件内守卫
export default {
  beforeRouteEnter(to, from, next) {
    // 在渲染该组件的对应路由被确认前调用
    // 不能获取组件实例 this，因为此时组件实例还没被创建
    next(vm => {
      // 通过 vm 访问组件实例
    })
  },
```

```
  beforeRouteUpdate(to, from, next) {
    // 在当前路由改变，但是该组件被复用时调用
    // 举例来说，对于带有动态参数的路径 /user/:id，在 /user/1 和 /user/2 之间
跳转的时候
    this.name = to.params.name
    next()
  },

  beforeRouteLeave(to, from, next) {
    // 导航离开该组件的对应路由时调用
    const answer = window.confirm('确定要离开吗？')
    if (answer) {
      next()
    } else {
      next(false)
    }
  }
}
```

路由守卫执行顺序详解
守卫执行顺序公式：

执行顺序=全局前置→路由独享→组件内→全局后置

具体执行流程： 1. **导航被触发** 2. **在失活的组件里调用 `beforeRouteLeave` 守卫** 3. **调用全局的 `beforeEach` 守卫** 4. **在重用的组件里调用 `beforeRouteUpdate` 守卫** 5. **在路由配置里调用 `beforeEnter`** 6. **解析异步路由组件** 7. **在被激活的组件里调用 `beforeRouteEnter`** 8. **调用全局的 `beforeResolve` 守卫** 9. **导航被确认** 10. **调用全局的 `afterEach` 钩子** 11. **触发 DOM 更新** 12. **在创建好的实例调用 `beforeRouteEnter` 守卫中传给 `next` 的回调函数**

---

7. 状态管理

7.1 Pinia状态管理

Pinia是Vue3的官方状态管理库，具有以下特点：

- 完整的TypeScript支持
- 轻量级（~1KB）
- 模块化设计
- DevTools支持

- 热更新支持

7.2 Store定义

```
// stores/counter.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  // 状态
  const count = ref(0)
  const lastAction = ref('')

  // 计算属性
  const doubleCount = computed(() => count.value * 2)
  const isEven = computed(() => count.value % 2 === 0)

  // 动作
  function increment() {
    count.value++
    lastAction.value = 'increment'
  }

  function decrement() {
    count.value--
    lastAction.value = 'decrement'
  }

  function reset() {
    count.value = 0
    lastAction.value = 'reset'
  }

  return {
    // 状态
    count,
    lastAction,
    // 计算属性
    doubleCount,
    isEven,
    // 动作
    increment,
    decrement,
    reset
  }
})
```

7.3 在组件中使用Store

```
<script setup lang="ts">
import { useCounterStore } from '@/stores/counter'

const counterStore = useCounterStore()

// 直接使用
console.log(counterStore.count)

// 使用storeToRefs保持响应性
import { storeToRefs } from 'pinia'
const { count, doubleCount } = storeToRefs(counterStore)

// 调用动作
function handleIncrement() {
  counterStore.increment()
}
</script>

<template>
  <div>
    <h2>Count: {{ counterStore.count }}</h2>
    <h2>Double: {{ counterStore.doubleCount }}</h2>
    <h2>Is Even: {{ counterStore.isEven }}</h2>
    <h2>Last Action: {{ counterStore.lastAction }}</h2>

    <button @click="counterStore.increment">+1</button>
    <button @click="counterStore.decrement">-1</button>
    <button @click="counterStore.reset">Reset</button>
  </div>
</template>
```

7.4 状态持久化

```
// stores/user.ts
import { defineStore } from 'pinia'
import { ref } from 'vue'

export const useUserStore = defineStore('user', () => {
  const user = ref<User | null>(null)
  const token = ref<string>('')

  function setUser(userData: User) {
    user.value = userData
    // 保存到localStorage
    localStorage.setItem('user', JSON.stringify(userData))
  }

  function setToken(tokenValue: string) {
    token.value = tokenValue
    localStorage.setItem('token', tokenValue)
  }

  function logout() {
    user.value = null
    token.value = ''
    localStorage.removeItem('user')
    localStorage.removeItem('token')
  }

  // 初始化时从localStorage恢复
  function initFromStorage() {
    const savedUser = localStorage.getItem('user')
    const savedToken = localStorage.getItem('token')

    if (savedUser) {
      user.value = JSON.parse(savedUser)
    }

    if (savedToken) {
      token.value = savedToken
    }
  }

  return {
    user,
```

```
    token,
    setUser,
    setToken,
    logout,
    initFromStorage
  }
})
```

7.5 模块化状态管理

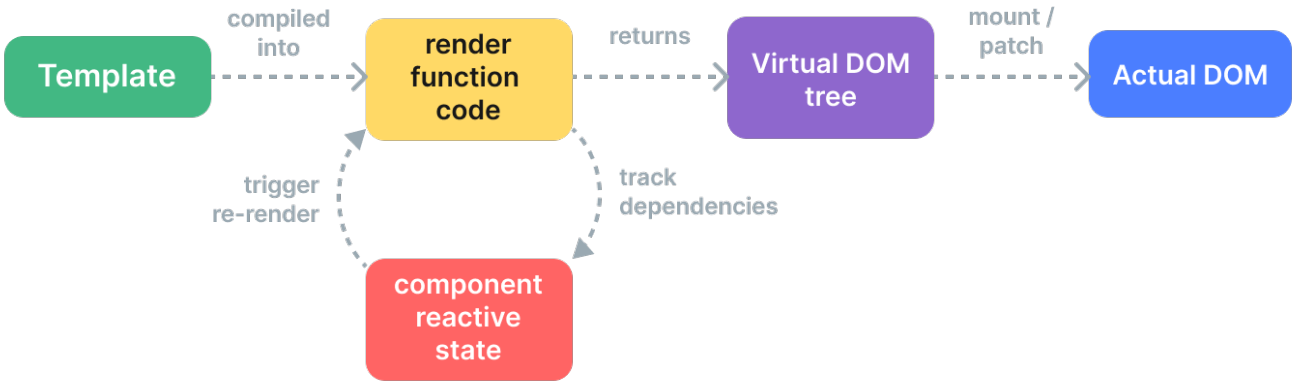
```
// stores/index.ts
import { useCounterStore } from './counter'
import { useUserStore } from './user'
import { useCartStore } from './cart'

// 统一导出所有store
export {
  useCounterStore,
  useUserStore,
  useCartStore
}

// 组合式API方式使用
export function useAllStores() {
  return {
    counter: useCounterStore(),
    user: useUserStore(),
    cart: useCartStore()
  }
}
```

---

8. 性能优化



8.1 响应式优化

8.1.1 shallowRef和markRaw

```
import { shallowRef, markRaw } from 'vue'

// 使用shallowRef避免深度响应
const largeObject = shallowRef({
  data: new Array(10000).fill(0).map((_, i) => ({
    id: i,
    value: Math.random()
  }))
})

// 使用markRaw标记非响应式对象
const config = markRaw({
  apiUrl: 'https://api.example.com',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json'
  }
})
```

8.1.2 readonly

```
import { reactive, readonly } from 'vue'

const original = reactive({ count: 0 })
const copy = readonly(original)

// 修改original会触发更新
original.count++

// 修改copy不会触发更新（警告）
copy.count++ // 警告: Set operation on key "count" failed: target is
readonly.
```

8.2 组件优化

8.2.1 异步组件

```
import { defineAsyncComponent } from 'vue'

// 基础异步组件
const AsyncComponent = defineAsyncComponent(() =>
  import('./components/HeavyComponent.vue')
)

// 带配置的异步组件
const AsyncComponentWithOptions = defineAsyncComponent({
  loader: () => import('./components/HeavyComponent.vue'),
  loadingComponent: LoadingComponent,
  errorComponent: ErrorComponent,
  delay: 200,
  timeout: 3000,
  suspensible: false
})
```

8.2.2 组件懒加载

```
// 路由懒加载
const routes = [
  {
    path: '/about',
    component: () => import('@/views/About.vue')
  }
]

// 组件懒加载
const LazyComponent = defineAsyncComponent({
  loader: () => import('@/components/HeavyComponent.vue'),
  loadingComponent: LoadingSpinner,
  errorComponent: ErrorComponent
})
```

8.3 虚拟滚动

```
<template>
  <div class="virtual-list" ref="container">
    <div class="virtual-list__spacer" :style="{ height: totalHeight +
'px' }">
      <div
        v-for="item in visibleItems"
        :key="item.id"
        class="virtual-list__item"
        :style="{ transform: `translateY(${item.offset}px)` }"
      >
        {{ item.content }}
      </div>
    </div>
  </div>
</template>

<script setup lang="ts">
import { ref, computed, onMounted, onUnmounted } from 'vue'

interface ListItem {
  id: number
  content: string
  height: number
  offset: number
}

const props = defineProps<{
  items: ListItem[]
  itemHeight: number
  containerHeight: number
}>()

const container = ref<HTMLElement>()
const scrollTop = ref(0)

const visibleItems = computed(() => {
  const startIndex = Math.floor(scrollTop.value / props.itemHeight)
  const endIndex = Math.min(
    startIndex + Math.ceil(props.containerHeight / props.itemHeight) +
1,
    props.items.length
  )
```

```
  return props.items.slice(startIndex, endIndex).map((item, index) =>
({
    ...item,
    offset: (startIndex + index) * props.itemHeight
  }))
})

const totalHeight = computed(() => props.items.length *
props.itemHeight)

function handleScroll() {
  if (container.value) {
    scrollTop.value = container.value.scrollTop
  }
}

onMounted(() => {
  if (container.value) {
    container.value.addEventListener('scroll', handleScroll)
  }
})

onUnmounted(() => {
  if (container.value) {
    container.value.removeEventListener('scroll', handleScroll)
  }
})
</script>
```

8.4 内存优化

8.4.1 事件清理

```
<script setup lang="ts">
import { onMounted, onUnmounted } from 'vue'

const handleResize = () => {
  console.log('窗口大小改变')
}

onMounted(() => {
  window.addEventListener('resize', handleResize)
})

onUnmounted(() => {
  window.removeEventListener('resize', handleResize)
})
</script>
```

8.4.2 定时器清理

```
<script setup lang="ts">
import { onMounted, onUnmounted } from 'vue'

let timer: NodeJS.Timeout

onMounted(() => {
  timer = setInterval(() => {
    console.log('定时任务执行')
  }, 1000)
})

onUnmounted(() => {
  if (timer) {
    clearInterval(timer)
  }
})
</script>
```

---

9. TypeScript支持

9.1 基础类型定义

```
// 基础类型
const count = ref<number>(0)
const message = ref<string>('Hello')
const isVisible = ref<boolean>(true)

// 复杂类型
interface User {
  id: number
  name: string
  email: string
  role?: 'admin' | 'user' | 'guest'
}

const user = ref<User>({
  id: 1,
  name: 'John Doe',
  email: 'john@example.com'
})

// 数组类型
const items = ref<number[]>([1, 2, 3])
const users = ref<User[]>([])

// 泛型类型
const data = ref<Record<string, any>>({})
const array = ref<Array<{ id: number; name: string }>>([])
```

9.2 组件Props类型定义

```
// 单文件组件
<template>
  <div>{{ title }}</div>
</template>

<script setup lang="ts">
interface Props {
  title: string
  subtitle?: string
  count?: number
  disabled?: boolean
  items: Array<{
    id: number
    name: string
  }>
}

const props = defineProps<Props>()

// 或者使用泛型方式
// const props = defineProps<Props>()
</script>
```

9.3 组件Emits类型定义

```
<script setup lang="ts">
interface Emits {
  update: [value: string]
  submit: [data: { name: string; email: string }]
  'custom-event': [param1: number, param2: string]
}

const emit = defineEmits<Emits>()

// 使用
function handleClick() {
  emit('update', 'new value')
}

function handleSubmit() {
  emit('submit', {
    name: 'John',
    email: 'john@example.com'
  })
}
</script>
```

9.4 组件Refs类型定义

```
<template>
  <input ref="inputRef" type="text" />
  <ChildComponent ref="childRef" />
</template>

<script setup lang="ts">
import { ref } from 'vue'
import type { ComponentPublicInstance } from 'vue'

// 基础元素refs
const inputRef = ref<HTMLInputElement>()

// 组件refs
const childRef = ref<ComponentPublicInstance>()

// 使用
function focusInput() {
  inputRef.value?.focus()
}

function callChildMethod() {
  childRef.value?.someMethod()
}
</script>
```

9.5 泛型组件

```
<script setup lang="ts">
interface Props<T> {
  items: T[]
  selectedItem?: T
  itemKey: keyof T
}

const props = defineProps<Props<any>>()

const emit = defineEmits<{
  'update:selectedItem': [item: any]
  select: [item: any, index: number]
}>()

// 泛型函数
function handleSelect<T>(item: T, index: number): T {
  emit('update:selectedItem', item)
  emit('select', item, index)
  return item
}
</script>
```

---

10. 测试与部署

10.1 单元测试

10.1.1 组件测试

```
// components/__tests__/Counter.spec.ts
import { mount } from '@vue/test-utils'
import Counter from '../Counter.vue'

describe('Counter', () => {
  it('renders properly', () => {
    const wrapper = mount(Counter, {
      props: { initialCount: 10 }
    })

    expect(wrapper.text()).toContain('10')
  })

  it('increments when button is clicked', async () => {
    const wrapper = mount(Counter)

    await wrapper.find('button').trigger('click')

    expect(wrapper.text()).toContain('1')
  })

  it('emits increment event', async () => {
    const wrapper = mount(Counter)

    await wrapper.find('button').trigger('click')

    expect(wrapper.emitted()).toHaveProperty('increment')
    expect(wrapper.emitted('increment')[0]).toEqual([1])
  })
})
```

10.1.2 Store测试

```
// stores/__tests__/counter.spec.ts
import { setActivePinia, createPinia } from 'pinia'
import { useCounterStore } from '../counter'

describe('Counter Store', () => {
  beforeEach(() => {
    setActivePinia(createPinia())
  })

  it('increments', () => {
    const counter = useCounterStore()

    expect(counter.count).toBe(0)

    counter.increment()

    expect(counter.count).toBe(1)
  })

  it('doubleCount computed property', () => {
    const counter = useCounterStore()

    counter.count = 5

    expect(counter.doubleCount).toBe(10)
  })
})
```

10.2 E2E测试

```
// cypress/e2e/app.cy.ts
describe('Vue3 App', () => {
  it('visits the app root url', () => {
    cy.visit('/')

    cy.contains('h1', 'Welcome to Your Vue.js + TypeScript App')
  })

  it('navigates to about page', () => {
    cy.visit('/')

    cy.get('a[href="/about"]').click()

    cy.url().should('include', '/about')
    cy.contains('h1', 'About')
  })
})
```

10.3 构建与部署

10.3.1 Vite配置

```
// vite.config.ts
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { resolve } from 'path'

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src')
    }
  },
  build: {
    outDir: 'dist',
    sourcemap: true,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['vue', 'vue-router', 'pinia'],
          utils: ['lodash', 'dayjs']
        }
      }
    }
  },
  server: {
    port: 3000,
    open: true
  }
})
```

10.3.2 部署配置

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v2

    - name: Setup Node.js
      uses: actions/setup-node@v2
      with:
        node-version: '16'

    - name: Install dependencies
      run: npm ci

    - name: Run tests
      run: npm run test:unit

    - name: Build
      run: npm run build

    - name: Deploy
      run: npm run deploy
```

总结

Vue3作为Vue.js的最新版本，在性能、开发体验和类型安全方面都有显著提升。本笔记涵盖了Vue3的核心概念和实际应用场景，包括：

1. **组合式API** - 提供更灵活的逻辑复用方式
2. **响应式系统** - 基于Proxy的新一代响应式实现
3. **组件系统** - 完整的组件通信和插槽机制

4. **模板语法** - 丰富的指令和绑定方式
5. **路由管理** - Vue Router 4的完整使用
6. **状态管理** - Pinia的现代化状态管理
7. **性能优化** - 各种优化策略和最佳实践
8. **TypeScript支持** - 完整的类型定义和泛型支持
9. **测试与部署** - 完整的测试和部署流程

掌握这些知识点，能够帮助开发者构建高质量、高性能的Vue3应用程序。

作者：MiniMax Agent
最后更新：2025-11-02